

Dynamic Programming

What is Dynamic Programming

Dynamic Programming (DP) is an optimization technique for problems that have overlapping subproblems and optimal substructure — meaning the problem can be broken into smaller subproblems whose solutions are reused rather than recomputed. DP is essentially 'recursion + memory'.

Two Core Properties

- Overlapping Subproblems: the same smaller subproblem is solved multiple times in a naive recursive solution (e.g. naive Fibonacci recalculates $\text{fib}(2)$ many times).
- Optimal Substructure: the optimal solution to the problem can be built from optimal solutions to its subproblems.

Two Approaches

- Top-Down (Memoization): write the recursive solution normally, but cache results in a dictionary/array so repeated calls return instantly.
- Bottom-Up (Tabulation): build the solution iteratively from the smallest subproblem upward, usually using an array, avoiding recursion overhead entirely.

Fibonacci - Naive vs Memoized vs Tabulated

```
# Naive -  $O(2^n)$ , recalculates everything
```

```
def fib_naive(n):
```

```
    if n <= 1:
```

```
        return n
```

```
    return fib_naive(n-1) + fib_naive(n-2)
```

```
# Memoized (top-down) -  $O(n)$ 
```

```
def fib_memo(n, cache={}):
```

```
    if n in cache:
```

```
        return cache[n]
```

```
    if n <= 1:
```

```
        return n
```

```
    cache[n] = fib_memo(n-1, cache) + fib_memo(n-2, cache)
```

```
    return cache[n]
```

```
# Tabulated (bottom-up) - O(n), O(1) space possible
```

```
def fib_tab(n):
```

```
    if n <= 1:
```

```
        return n
```

```
    prev, curr = 0, 1
```

```
    for _ in range(2, n + 1):
```

```
        prev, curr = curr, prev + curr
```

```
    return curr
```

Classic DP Problems

- 0/1 Knapsack — maximize value within a weight limit, each item used at most once.
- Longest Common Subsequence (LCS) between two strings.
- Longest Increasing Subsequence (LIS).
- Coin Change — minimum coins to make a target amount.
- Edit Distance — minimum operations to convert one string to another.
- House Robber — maximum sum without picking adjacent elements.

How to Recognize a DP Problem

- The problem asks for a maximum, minimum, or count of ways to do something.
- A brute-force/recursive solution would repeat the same subproblem many times.
- The problem involves making a sequence of choices where each choice affects future options.